Rocket Digital Twin

Digital twin of a launch vehicle delivering a satellite to low Earth orbit (LEO)
Technical Documentation

SpatiumCavum Team | Mateusz Bala
github.com/MateuszBala/Spaceshield_Hack_2026_PTR_DLD_Digital_Twin_Mission
Hackathon Space Shield 2026

Table of Contents

1. Launch and Run

The system consists of two processes started in parallel: the backend (API + physics engine, Python) and the frontend (browser interface, Node). The procedure below is step-by-step.

1.1 Requirements

- Python 3.13 and `uv` (Python environment and package manager).
- Node.js 18+ and `npm` (for the frontend).
- Operating system: Linux / macOS / WSL (validated on Debian).

1.2 Backend - API with physics engine

From the repository root directory (`uv` workspace includes `contracts`, `physics-engine`, `api`):

```
# 1) install workspace dependencies (one-time)
uv sync

# 2) start API server on port 8000
uv run uvicorn dt_api.app:app --port 8000
```

After startup, the server is available at `http://localhost:8000`. Interactive API documentation (Swagger) is available at `http://localhost:8000/docs`, where you can invoke the `/simulate` endpoint directly.

1.3 Frontend - browser interface

In a separate terminal, from the `frontend/` directory:

```
# 1) install npm dependencies (one-time)
npm install

# 2) start development server
npm run dev
```

The interface is available at `http://localhost:5173`. The frontend communicates with the API via proxy (`/api/* -> localhost:8000`).

1.4 Startup verification

After opening `http://localhost:5173`, the top-right status badge indicates the current data source:

- **API LIVE - ENGINE ✓** - frontend connected to the API; results come from the real physics engine (target state).
- **API LIVE - ENGINE stub** - API responds, but engine is unavailable (fallback result is returned).
- **API offline** - no API connection; interface runs on synthetic data (offline demo mode).

The state **API LIVE - ENGINE ✓** confirms that the whole chain (interface -> API -> engine -> verdict) works correctly.

2. Interface and Usage

The interface implements the Digital Twin loop: the user sets rocket parameters, starts simulation with the *Recompute* button, and receives an orbital verdict together with full flight visualization. Every change can be repeated freely - this is the core of a digital twin: experimenting on the model before a real flight.

2.1 Main view: parameters, verdict, sequence, telemetry

Figure 1: Main view - parameter panel (left), verdict card with orbital elements, mission sequence, and telemetry panel.

Parameter panel (left column). Each rocket stage (S1, S2, S3) is presented as a card with input fields: thrust [N], specific impulse (Isp) [s], burn time [s], and dry mass [kg]. Under the inputs, the UI shows derived values: propellant mass, wet mass, mass fraction (η_m), and mass flow rate. Below, payload mass and total liftoff mass are displayed. The Δv_{eff} indicator at the top is an estimated velocity budget. At the bottom, the *Recompute* button starts simulation.

Verdict card. After simulation, it displays a clear outcome: **ORBIT ACHIEVED** or **NO ORBIT**, together with a natural-language reason (for example: "LEO achieved: ε, e, perigee"). The *Flight summary* section reports maximum altitude, flight time, and Max-Q. The *Orbital elements* section shows perigee, apogee, eccentricity, semi-major axis, period, and specific energy. The verdict is derived from Keplerian elements (see Section 3), not from instantaneous altitude.

Mission sequence. A timeline of milestones in a broadcast style: LIFTOFF, MECO (main stage burnout), STAGE SEP (separation), SECO (second stage burnout), APOGEE, ORBITAL INSERTION. Each event shows time, altitude, and speed.

Telemetry panel (HUD). A prominent live-style readout: speed [km/h], altitude [km], g-load [g], mass [t], and dynamic pressure [Pa]. A time scrubber under the panel allows navigation to any flight instant - readouts and phase marker (for example INSERTION, Stage 3) update accordingly.

2.2 Flight path visualization in space

Figure 2: Flight path in inertial frame (X, Y) - Earth at center, ascent/insertion track (solid line), and full Keplerian orbit (dashed line).

The *Flight Path - Inertial Frame (X, Y)* chart shows the actual trajectory around Earth (origin at (0, 0)). The solid line is the ascent/insertion path reconstructed from telemetry. The dashed line is the full orbit computed analytically from Keplerian elements - it shows the post-insertion orbit even though simulation stops at insertion. Events (launch, separations, apogee) are marked. This view proves that the system models full orbital geometry, not just the traversed trajectory segment.

2.3 Side profile - atmospheric ascent segment

Figure 3: Side profile - altitude as a function of downrange distance (up to 400 km), with atmospheric segment and stage separation events.

The *Side Profile* chart presents the early flight stage: altitude relative to Earth surface during atmospheric exit (up to 400 km). The trajectory curvature (gravity turn) and stage separation events are visible - this is the flight segment where aerodynamic drag is significant.

2.4 Telemetry charts over time

Figure 4: 2D trajectory (altitude and speed over time) and g-force profile with characteristic saw-tooth behavior.

2D trajectory - altitude and speed. Two curves over time: altitude (left axis) and speed (right axis). Stage burnout points and apogee are visible. During ballistic coast, altitude grows toward apogee while speed decreases to a minimum at apogee - consistent with energy conservation.

G-force profile. Acceleration [g] over time. The characteristic saw-tooth pattern appears naturally: during a stage burn, acceleration rises (mass decreases at nearly constant thrust), drops near zero at burnout/separation, then jumps at ignition of the next stage. The final jump corresponds to upper-stage burn (apogee circularization). G-force peaks align with separation events, confirming telemetry consistency with mission sequence.

Dynamic pressure (Q). Quantity `Q = 1/2 * rho * v^2` over time - critical for structural loads. A clear early-flight maximum (Max-Q) indicates peak aerodynamic stress; then Q decays rapidly with atmospheric thinning.

2.5 Usage pattern: the experiment loop

1. Set or modify a rocket parameter in the panel (for example stage burn time, thrust, payload mass).
2. Click *Recompute*. The system invokes the physics engine via API.
3. Read the verdict (orbit achieved or not), plus updated trajectory, sequence, and telemetry.
4. Repeat with another value to evaluate design-decision impact on mission success.

3. Physical Foundations

The simulation engine is based on two-body orbital mechanics. References: MIT 16.346 Astrodynamics lectures (Keplerian elements), and *Space Mission Analysis and Design* (SMAD) for physical constants and orbital thresholds. The challenge allows simplified models; every simplification here is an intentional engineering decision.

3.1 State vector and reference frame

The full flight - from liftoff to orbit - is solved in a single inertial Cartesian frame with Earth center at the origin. Rocket state is defined by five values:

$$s = [x, y, v_x, v_y, m]$$

where (x, y) is position, (v_x, v_y) is velocity, and m is instantaneous mass. Flight is modeled in the orbital plane (2D) - the orbit is planar and key quantities are derivable from this state. Stage separation is a mass-only discontinuity; position and velocity remain continuous.

3.2 Forces and equations of motion

During powered atmospheric flight, three forces act on the vehicle:

Gravity toward Earth center:

$$g = -(mu / r^3) * (x, y), \text{ where } r = \text{sqrt}(x^2 + y^2)$$

with mu = 3.986 x 10^{14} m^3/s^2.

Thrust along vehicle axis, controlled by flight-angle profile. Mass decreases with propellant flow:

$$dm/dt = -F_{thrust} / (Isp * g_0)$$

where g_0 = 9.80665 m/s^2.

Aerodynamic drag opposite to velocity:

$$D = 1/2 * rho(h) * v^2 * C_d * A, \text{ where } rho(h) = rho_0 * \exp(-h/H)$$

where rho(h) is atmospheric density decreasing exponentially with altitude, C_d is drag coefficient, and A is reference area.

3.3 Orbital verdict from Keplerian elements

This is the core of the model. Reaching orbital altitude does not imply orbit - a vehicle may be at 400 km with upward-directed velocity and still fall back. Orbit is determined from trajectory elements computed from the state vector at insertion.

Specific orbital energy:

$$\text{epsilon} = v^2/2 - \mu/r$$

Bound-orbit condition: epsilon < 0.

Semi-major axis:

$$a = -\mu / (2 * \text{epsilon})$$

Eccentricity vector (Laplace vector; $h = x*v_y - y*v_x$):

$$e_x = v_y*h/\mu - x/r, \quad e_y = -v_x*h/\mu - y/r, \quad e = \text{sqrt}(e_x^2 + e_y^2)$$

Eccentricity e determines orbital elongation (e = 0 means circular orbit).

Perigee altitude:

$$h_p = a * (1 - e) - R_{Earth}, \text{ where } R_{Earth} = 6378 \text{ km}$$

Success criterion (SMAD, ch. 7.3 thresholds):

$$\text{epsilon} < 0 \text{ AND } h_p > 200 \text{ km AND } e < 0.25$$

That is: bound orbit, perigee above dense atmosphere (stable LEO), and limited eccentricity (no highly elongated extreme orbit).

3.4 Insertion maneuver (gravity turn + Hohmann)

Orbital ascent is two-stage. First, the rocket performs a gravity turn to gradually convert vertical climb into horizontal velocity. Then, after lower-stage burnout, a ballistic coast phase carries the vehicle toward apogee. The upper stage ignites near apogee for circularization (Hohmann-style insertion), adding horizontal speed, raising perigee, and closing the orbit. This is why the final g-force jump corresponds to upper-stage burn, while speed reaches a minimum at apogee right before this burn.

3.5 Propulsion model consistency

Remaining quantities are derived from stage parameters (thrust, Isp, burn time), preventing physically inconsistent configurations:

$$\dot{m} = F_{thrust} / (Isp * g_0), \quad m_{propellant} = \dot{m} * t_{burn}$$

Velocity budget for a given stack is estimated via the Tsiolkovsky equation per stage:

$$\Delta v = Isp * g_0 * \ln(m_{initial} / m_{final})$$

Specific impulse is constrained to `Isp <= 455 s` (hard chemical-propulsion limit). Typical LEO budget is about 9.4 km/s (about 7.7 km/s orbital speed + about 1.75 km/s losses).

3.6 What is modeled vs. what is simplified

Modeled: motion in gravitational field (two-body), thrust and stage-wise propellant depletion, atmospheric drag with exponential density model, staging, Max-Q, Keplerian verdict from full elements, and 1-4 stage configurations.

Simplified by design: 2D flight instead of 3D (planar orbit), flight-angle profile instead of full guidance optimization, exponential atmosphere instead of layered model, and omission of Earth rotation, J2, and shape-dependent drag. These simplifications affect trajectory fidelity but not the orbital-mechanics correctness used for verdicting.

4. Architecture and Implementation

4.1 System layers

The system is a monorepo split into separate packages around one shared data contract:

- **Physics engine** (`dt_physics`, NumPy/SciPy) - pure numerics, function `simulate(SimRequest) -> SimResult`. No network/interface concerns; easy to test and reuse.
- **API** (`dt_api`, FastAPI) - thin HTTP layer. Accepts request, calls engine, returns result. No physics business logic.
- **Frontend** (React + Vite + TypeScript) - user interface; communicates with backend only via HTTP.
- **Data contract** (`dt_contracts`, Pydantic) - single source of truth for data shape (inputs, outputs, telemetry, orbital elements). Frontend types are generated from OpenAPI, preventing frontend/backend drift.

Each layer has one responsibility and a clear boundary, enabling parallel development. The HTTP boundary exists where it is natural (browser -> server). Inside the backend, API calls the engine through direct function invocation.

4.2 Three phases as numerical regimes

Phase	Dominant physics	Termination event
Ascent	Thrust + gravity + drag rho(h)	Max-Q, separations, drag threshold
Insertion	Thrust + gravity (drag negligible)	Burnout / orbital-speed target
Orbit	Two-body, no thrust	Keplerian verdict

Table 1: Three flight phases and their numerical characteristics.

Phase transitions are detected as solver events (root crossings), not hard-coded time thresholds, so transition timing does not depend on integration step density. For example, Max-Q is detected at the zero crossing of dQ/dt.

4.3 Key design decisions

- **Custom simulator instead of OpenRocket** - OpenRocket does not model orbital insertion to LEO as required here. A custom engine gave full control over orbital mechanics and verdict logic.
- **Verdict from Keplerian elements, not altitude** - the most important physical decision, distinguishing this project from an altitude calculator.
- **Graceful degradation** - optional functions (for example AI optimization) do not block the core path; API reports availability and UI hides unavailable options.
- **Delta-v fallback path** - if full integration fails, a defined fallback path exists (Tsiolkovsky budget), rather than ad-hoc behavior.

4.4 API endpoints

- `POST /simulate` - mission simulation: SimRequest -> SimResult.
- `GET /capabilities` - engine and AI module availability, contract version.
- `GET /health` - server status.
- `POST /optimize` - optimization (returns 503 when AI module is unavailable).

4.5 References

- MIT OpenCourseWare 16.346 Astrodynamics - two-body problem, eccentricity vector, energy integral, Kepler laws.
- *Space Mission Analysis and Design* (SMAD), 3rd ed. - Earth constants, g0, specific impulse and mass-fraction ranges, orbital thresholds.